

Affordmed Style Mock Assessment

Duration: 3 Hours

Candidates can choose Backend, Frontend, or Full Stack track.

Problem 1: User Activity Analytics Service

Objective

Develop a REST API microservice:

GET /analytics

The API accepts multiple URLs through query parameters.

Example:

/analytics?url=<http://server1/activity>&url=<http://server2/activity>

Each URL returns:

```
{
  "users": [
    { "username": "john", "duration": 120 },
    { "username": "mary", "duration": 90 }
  ]
}
```

Expected Processing

The service must:

1. Validate URLs
2. Fetch data from all valid URLs
3. Ignore invalid URLs
4. Ignore slow responses
5. Merge all user activity records
6. Calculate total active users
7. Calculate average session duration
8. Identify the most active user
9. Return the final analytics summary

Example Response

```
{
  "totalUsers": 15,
  "averageDuration": 96.5,
  "mostActiveUser": { "username": "john", "duration": 420 }
}
```



Skills Evaluated

Backend Development

- REST APIs
- Query Parameters
- Express.js

Concurrency

- Promise.all()
- Async/Await
- Timeout Handling

Data Processing

- Object Traversal
- Aggregation
- Statistics Calculation

Performance Engineering

- Fast Response Time
- Parallel API Calls

Difficulty Level

Medium

Industry Relevance

High

Problem 2: Auto Suggest Service

Objective

Develop a REST API:

GET /suggestions

Example:

/suggestions?query=cha

Server contains a predefined dictionary:

```
["chair", "change", "character", "chart", "camera", "card"]
```

Expected Processing

1. Accept search query
2. Find all matching words
3. Return matching words that start with the given query
4. Support optional limit parameter

Example:

/suggestions?query=cha&limit=3



Response:

```
{ "query": "cha", "suggestions": ["chair", "change", "character"] }
```

If no matches exist:

```
{ "query": "xyz", "suggestions": [] }
```

Skills Evaluated

String Processing

- Prefix Search
- String Matching

Data Structures

- Arrays
- Filtering Logic

Algorithm Design

- Search Optimization
- Prefix Based Retrieval

Difficulty Level

Medium

Industry Relevance

High

Full Stack Assessment

Question 1

Build an Event Discovery Aggregation Microservice.

Features

1. Register with external APIs
2. Fetch events from multiple providers
3. Aggregate event information
4. Apply filtering
5. Apply sorting
6. Apply pagination
7. Expose public APIs

Sample Event

```
{ "eventName": "React Summit", "city": "Chennai", "price": 1500, "rating": 4.5 }
```

**Example APIs**

GET /events

GET /events?city=Chennai

GET /events?sort=price

GET /events?page=1&limit=10

GET /events?rating=4

Skills Evaluated

- REST APIs
- API Integration
- Pagination
- Sorting
- Filtering
- Aggregation Logic
- Backend Architecture

Question 2

Build a React Frontend.

Requirements

- React / Next.js
- Mobile Responsive Design
- Desktop Responsive Design
- Consume APIs from Question 1
- No direct calls to vendor APIs

Features

- Event Listing
- Search Events
- Filter by City
- Filter by Rating
- Sort by Price
- Pagination
- Responsive UI

Skills Evaluated

- React Fundamentals
- API Consumption
- Responsive Design
- State Management
- Component Architecture



- UI Engineering

Overall Assessment Difficulty

Backend Track: Medium

Frontend Track: Medium

Full Stack Track: Medium-High

Primary Focus

Engineering Implementation + API Integration + Practical Problem Solving

Secondary Focus

DSA and OOP Concepts